

UNIVERSITÉ PARIS DAUPHINE – PSL

LAMSADE

Master – Module : **Qualité de Données, Data Wrangling**

Encadrant : **Khalid Belhajjame**

Découverte de Pattern Functional Dependencies

Approche Classique vs. Agentique (IA)

SOLIMAN Omar **DERRADJI** Fahd

HADDAR Djena **TSOULI** Abderrahmane

KARAOUANE Guillaume

Année universitaire 2025 – 2026

Table des matières

1	Introduction	3
1.1	Contexte et motivations	3
1.2	Objectifs du projet	3
1.3	Organisation du rapport	3
2	Concepts théoriques	3
2.1	Dépendances fonctionnelles classiques	3
2.2	Pattern Functional Dependencies (PFDs)	4
2.3	PFDs approximatives : support et confiance	4
2.4	Transformations utilisées	5
3	Architecture du système	5
3.1	Vue d'ensemble	5
3.2	Technologies utilisées	6
3.3	Datasets	6
4	Approche classique	6
4.1	Pipeline en 5 étapes	6
4.1.1	Étape 1 : Extraction de patterns	7
4.1.2	Étape 2–3 : Génération de candidats	7
4.1.3	Étape 4 : Validation	7
4.1.4	Étape 5 : Généralisation	8
5	Approche agentique	8
5.1	Principe général	8
5.2	Intégration LLM via Ollama	8
5.3	Workflow 1 : Feature-Enriched Discovery	8
5.4	Workflow 2 : Guided Search	9
5.5	Comparaison des workflows	9
6	Résultats expérimentaux	9
6.1	Paramètres expérimentaux	9
6.2	Résultats de l'approche classique	10
6.2.1	Exemples de PFDs découvertes	10
6.3	Résultats de l'approche agentique	11
6.3.1	Comparaison globale sur t1.csv	11
6.3.2	Analyse de la réduction de l'espace de recherche	11
6.3.3	Suggestions des agents	11
6.3.4	Candidats prioritaires (Workflow 2)	12
6.4	Comparaison Mistral vs. Llama	12
7	Discussion	13
7.1	Avantages de l'approche agentique	13
7.2	Limites et compromis	13
7.3	Analyse du compromis qualité/exhaustivité	13
7.4	Reproductibilité	14

8 Conclusion	14
A Annexe : Guide d'utilisation	15
A.1 Installation	15
A.2 Exécution	15
B Annexe : Structure du code	15

1 Introduction

1.1 Contexte et motivations

La qualité des données est un enjeu majeur dans les systèmes d'information modernes. Les **dépendances fonctionnelles** (FDs) constituent un outil fondamental pour détecter les anomalies, nettoyer les données et comprendre la structure sémantique d'un jeu de données. Cependant, les FDs classiques comparent des **valeurs entières** et ne capturent pas les régularités au niveau des *patterns* (préfixes, tokens, sous-chaînes).

Les **Pattern Functional Dependencies** (PFDs) généralisent les FDs en utilisant des transformations sur les valeurs. Par exemple, la règle `prefix(zip, 3) → city` exprime que les 3 premiers chiffres d'un code postal déterminent généralement la ville — une régularité invisible aux FDs classiques.

1.2 Objectifs du projet

Ce projet poursuit deux objectifs principaux :

1. **Implémenter un algorithme classique** de découverte de PFDs, suivant un pipeline en 5 étapes : extraction de patterns, groupement, génération de candidats, validation et généralisation.
2. **Concevoir et implémenter une approche agentique** utilisant des LLMs (Large Language Models) locaux pour guider intelligemment la recherche de PFDs, et comparer les résultats avec l'approche classique.

Nous comparons deux LLMs locaux — **Mistral 7B** et **Llama 3.1 8B** — exécutés via Ollama, sur 14 datasets réels couvrant trois domaines : gouvernance de données, chimie/biologie moléculaire et validation.

1.3 Organisation du rapport

La section 2 présente les concepts théoriques fondamentaux. La section 3 décrit l'architecture du système. La section 4 détaille l'approche classique. La section 5 présente l'approche agentique. La section 6 analyse les résultats expérimentaux. La section 7 discute les enseignements et limites. La section 8 conclut le rapport.

2 Concepts théoriques

2.1 Dépendances fonctionnelles classiques

Définition – Dépendance Fonctionnelle

Soit une relation R sur un schéma $\mathcal{S}$. Une **dépendance fonctionnelle** $X \rightarrow Y$ (où $X, Y \subseteq \mathcal{S}$) tient si et seulement si :

$$\forall t_1, t_2 \in R : t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y]$$

En d'autres termes, si deux tuples ont la même valeur sur X , ils doivent avoir la même valeur sur Y . Par exemple, dans une table d'employés, $\text{Department} \rightarrow \text{Department Name}$ signifie que le code du département détermine de manière unique son nom complet.

Limites des FDs classiques :

- Elles comparent des **valeurs entières** uniquement.
- Elles ne détectent pas que les prénoms commençant par « John » sont masculins.
- Elles ne capturent pas que les codes postaux avec le même préfixe de 3 chiffres correspondent à la même ville.

2.2 Pattern Functional Dependencies (PFDs)

Définition – Pattern Functional Dependency

Une PFD $R(X \rightarrow Y, T_p)$ généralise les FDs en utilisant un **tableau de patterns** T_p . Elle tient si :

$$\forall t_1, t_2 \in R : T_p(t_1[X]) = T_p(t_2[X]) \Rightarrow t_1[Y] = t_2[Y]$$

où T_p est une transformation appliquée aux valeurs de X .

Exemples concrets :

- `first_token(name) → gender` : le prénom détermine le genre
- `prefix(zip, 3) → city` : les 3 premiers chiffres du code postal déterminent la ville
- `first_token(ref_id) → ref_type` : le premier token d'un identifiant de référence détermine son type (PubMed, DailyMed, etc.)

2.3 PFDs approximatives : support et confiance

Les données réelles contiennent du **bruit**. Une PFD ne tient donc pas à 100%. On utilise deux métriques pour quantifier la qualité d'une PFD approximative :

Métriques de qualité

Support : nombre de tuples couverts par les groupes formés par le pattern X :

$$\text{support}(X) = \sum_{g \in \text{Groups}(X)} |g|$$

Confiance : proportion de tuples « cohérents » (qui respectent la dépendance) :

$$\text{conf}(X \rightarrow Y) = \frac{\sum_{g \in \text{Groups}(X)} \text{count}(\text{majority}_Y(g))}{\sum_{g \in \text{Groups}(X)} |g|}$$

Bruit (Noise) : taux de violation, complémentaire de la confiance :

$$\text{noise} = 1 - \text{conf}(X \rightarrow Y)$$

On ne retient une PFD que si elle satisfait deux seuils :

- support $\geq K$ (dans nos expériences, $K = 5$)
- confidence $\geq \theta$ (dans nos expériences, $\theta = 0.85$)

2.4 Transformations utilisées

Notre système supporte 7 types de transformations, résumés dans le tableau 1.

TABLE 1 – Les 7 transformations supportées par le système

Transformation	Notation	Exemple	Usage typique
Identity	<code>identity(col)</code>	« Chicago » $\rightarrow$ « Chicago »	Colonnes catégorielles
Prefix	<code>prefix(col, k)</code>	<code>prefix("90012", 3)</code> $\rightarrow$ « 900 »	Codes postaux
Suffix	<code>suffix(col, k)</code>	<code>suffix("report.pdf", 3)</code> $\rightarrow$ « pdf »	Extensions
First token	<code>first_token(col)</code>	« John Smith » $\rightarrow$ « John »	Prénoms
Last token	<code>last_token(col)</code>	« John Smith » $\rightarrow$ « Smith »	Noms de famille
Numeric prefix	<code>numeric_prefix(col, k)</code>	« ZIP-90012 » $\rightarrow$ « 900 »	Données brutes
Length	<code>length(col)</code>	« Hello » $\rightarrow$ « 5 »	Longueur de chaînes

3 Architecture du système

3.1 Vue d'ensemble

Le projet est organisé en modules Python indépendants et réutilisables, totalisant environ **1 400 lignes de code** réparties sur 14 fichiers. L'architecture suit une séparation nette entre le cœur algorithmique, l'intégration LLM et les scripts d'expérimentation.

TABLE 2 – Organisation des modules du projet

Module	Lignes	Rôle
<code>data_loader.py</code>	45	Chargement CSV et introspection du schéma
<code>pattern_extraction.py</code>	152	Définition et application des 7 transformations
<code>pattern_grouping.py</code>	38	Groupement des tuples par pattern
<code>candidate_generation.py</code>	43	Génération des paires candidats (T_X, Y)
<code>validation.py</code>	110	Calcul du support et de la confiance
<code>generalization.py</code>	87	Fusion et déduplication des PFDs
<code>classical_pipeline.py</code>	87	Orchestration du pipeline en 5 étapes
<code>llm_agent.py</code>	216	Intégration Mistral/Llama via Ollama
<code>agentic_workflow.py</code>	261	Workflows agentiques 1 et 2
<code>evaluation.py</code>	124	Métriques et comparaison
<i>Total src/</i>	<i>1 163</i>	
<code>run_classical.py</code>	60	Script d'expérimentation classique
<code>run_agentic.py</code>	95	Script d'expérimentation agentique
<code>compare_results.py</code>	103	Script de comparaison
<i>Total général</i>	<i>1 421</i>	

3.2 Technologies utilisées

TABLE 3 – Stack technologique

Technologie	Usage
Python 3.10+	Langage principal
pandas	Manipulation de données tabulaires
Ollama	Serveur de LLMs locaux (REST API sur <code>localhost:11434</code>)
Mistral 7B	LLM local – 7 milliards de paramètres (~ 4.4 Go)
Llama 3.1 8B	LLM local – 8 milliards de paramètres (~ 4.9 Go)

Le choix de LLMs locaux (via Ollama) nous permet de travailler **sans quota**, **sans clé API**, et **hors-ligne**. Les deux modèles tournent confortablement sur un MacBook Air M2 avec 16 Go de RAM.

3.3 Datasets

Nous utilisons **14 fichiers CSV** répartis en 3 catégories, représentant un total de **46 351 lignes** et **103 colonnes** (tableau 4).

TABLE 4 – Datasets utilisés

Catégorie	Fichier	Lignes	Colonnes	Domaine
pfd_validation/	US_Phone_Code.csv	51	3	Téléphone US
	t1.csv	9 101	9	Employés gouvernementaux
	t2.csv	3 502	13	Entreprises commerciales
	t3.csv	1 077	9	Licences commerciales
CHE/	mechanism_refs.csv	9 536	5	Références pharmaceutiques
	metabolism_refs.csv	2 409	5	Métabolisme médicaments
	protein_classification.csv	858	7	Taxonomie protéines
	research_companies.csv	812	5	Entreprises pharma
	variant_sequences.csv	1 200	7	Variants de séquences
DGOV/	570-1.csv	9 101	9	Employés gouvernementaux
	10492-1.csv	1 077	9	Licences commerciales
	10642-1.csv	920	6	Agences d'emploi
	6339-1.csv	306	7	Statistiques criminalité
	6397-1.csv	6 704	9	Données démographiques

4 Approche classique

4.1 Pipeline en 5 étapes

L'approche classique suit un pipeline séquentiel en 5 étapes, illustré par la figure 1.

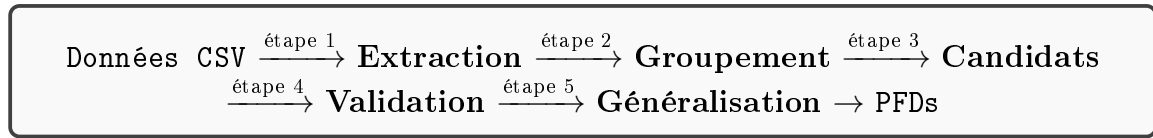


FIGURE 1 – Pipeline classique de découverte de PFDs

4.1.1 Étape 1 : Extraction de patterns

Pour chaque colonne du dataset, le système génère automatiquement les transformations pertinentes. La fonction `generate_default_transformations()` analyse chaque colonne et produit :

- `identity` pour toutes les colonnes.
- `first_token` et `last_token` si la colonne contient des espaces ou virgules (noms, adresses).
- `prefix(col, k)` pour $k \in [1, \min(\bar{L}, 5)]$ où $\bar{L}$ est la longueur moyenne des valeurs.
- `numeric_prefix(col, k)` si la colonne contient des chiffres.
- `length` pour toutes les colonnes.

Exemple : sur `t1.csv` (9 colonnes), cette étape génère environ **60 transformations**.

4.1.2 Étape 2–3 : Génération de candidats

Le système forme le produit cartésien des transformations et des colonnes cibles, en excluant les auto-dépendances triviales (`identity(X) → X`). Pour `t1.csv`, cela produit **~544 candidats**.

4.1.3 Étape 4 : Validation

L’algorithme de validation est le cœur du système. Pour chaque candidat $T(X) \rightarrow Y$:

Algorithm 1 Validation d’un candidat PFD

Require: DataFrame D , transformation T , colonne cible Y , seuils K, θ

Ensure: PFDResult ou $\perp$

```

1:  $groups \leftarrow \text{groupBy}(D, T)$  {Grouper par pattern}
2: Filtrer les groupes avec  $|g| < K$ 
3:  $total \leftarrow 0$ ,  $consistent \leftarrow 0$ 
4: for chaque groupe  $g$  dans  $groups$  do
5:    $counts \leftarrow \text{valueCounts}(g[Y])$ 
6:    $majority \leftarrow \max(counts)$ 
7:    $total \leftarrow total + |g|$ 
8:    $consistent \leftarrow consistent + majority$ 
9: end for
10:  $conf \leftarrow consistent/total$ 
11: if  $total \geq K$  and  $conf \geq \theta$  then
12:   return PFDResult( $T, Y, total, conf$ )
13: else
14:   return  $\perp$ 
15: end if
  
```

Complexité : $\mathcal{O}(C \times n \log n)$ où C est le nombre de candidats et n le nombre de lignes.

4.1.4 Étape 5 : Généralisation

La généralisation élimine les PFDs redondantes via trois stratégies :

1. **Préfixes** : parmi `prefix(col, 1)`, `prefix(col, 2)`, ..., garder le plus court avec une confiance $\geq$ (meilleure confiance -0.05).
2. **Subsomption** : si `identity(col)` a une bonne confiance, elle subsume les transformations sur la même colonne.
3. **Déduplication** : éliminer les doublons par type de transformation et colonne cible.

Exemple : sur `t1.csv`, la généralisation réduit **143 PFDs valides** à **85 PFDs généralisées**.

5 Approche agentique

5.1 Principe général

L'idée centrale est de combiner la **rigueur algorithmique** du pipeline classique avec le **raisonnement sémantique** d'un LLM. Le modèle de langage analyse les noms de colonnes et des échantillons de valeurs pour suggérer des transformations *pertinentes*, réduisant ainsi l'espace de recherche.

Nous implémentons deux workflows distincts, de complexité croissante.

5.2 Intégration LLM via Ollama

Les deux LLMs sont appelés via l'API REST d'Ollama (`POST /api/generate`). Chaque appel envoie un prompt structuré et reçoit une réponse en JSON. Les paramètres utilisés :

- `temperature` = 0.1 (réponses déterministes, peu de variabilité)
- `num_predict` = 4096 (longueur maximale de la réponse)
- `stream` = false (réponse complète en une fois)

5.3 Workflow 1 : Feature-Enriched Discovery

Workflow 1 – Feature-Enriched Discovery

Table $R \rightarrow$ **Agent LLM** $\rightarrow$ Transformations suggérées $\rightarrow$ **Algo classique** $\rightarrow$ PFDs

Fonctionnement détaillé :

1. **Analyse du schéma** : le système extrait pour chaque colonne le nombre de valeurs uniques et 5 échantillons de valeurs.
2. **Appel LLM** : un prompt structuré demande au LLM de suggérer des transformations pertinentes parmi les 7 types disponibles, avec justification.
3. **Parsing** : les suggestions JSON sont converties en objets `Transformation`.

4. **Pipeline classique** : les étapes 2 à 5 du pipeline classique sont exécutées avec les transformations suggérées par l'agent.

Exemple de suggestion (Mistral sur t1.csv) :

```

1 {
2   "name": "first_token",
3   "column": "Full Name",
4   "params": {},
5   "reason": "Le premier mot du nom complet est le prenom"
6 }
```

5.4 Workflow 2 : Guided Search

Workflow 2 – Guided Search

Table $R \rightarrow$ **Agent LLM** $\rightarrow$ Transformations + Candidats prioritaires $\rightarrow$
Algo : valide $\rightarrow$ PFDs

Le Workflow 2 va plus loin : après avoir suggéré les transformations, l'agent **priorise les paires** (T_X, Y) les plus prometteuses. Seuls les candidats jugés **high** ou **medium** par le LLM sont validés.

Exemple de priorisation (Llama sur t1.csv) :

```

1 {
2   "x_transformation": "prefix(Full Name, 1)",
3   "y_column": "Gender",
4   "priority": "high",
5   "reason": "Le premier token du nom determine le sexe"
6 }
```

5.5 Comparaison des workflows

TABLE 5 – Comparaison structurelle des trois approches

	Classique	Workflow 1	Workflow 2
Choix des transformations	Automatique	LLM	LLM
Sélection des candidats	Exhaustive	Exhaustive	LLM
Validation	Algorithmique	Algorithmique	Algorithmique
Généralisation	Algorithmique	Algorithmique	Algorithmique
Appels LLM	0	1	2

6 Résultats expérimentaux

6.1 Paramètres expérimentaux

Toutes les expériences utilisent les mêmes paramètres :

- Support minimum : $K = 5$
- Confidence minimum : $\theta = 0.85$
- Longueur maximale des préfixes : 4
- Machine : MacBook Air M2, 16 Go RAM

6.2 Résultats de l’approche classique

Le tableau 6 présente les résultats de l’approche classique sur les 14 datasets.

TABLE 6 – Résultats de l’approche classique sur les principaux datasets

Dataset	Candidats	Valides	Généralisées	Temps (s)	Conf. max
t1.csv	544	143	85	8.56	1.000
t2.csv	1 428	327	190	12.73	1.000
t3.csv	704	105	63	2.78	1.000
mechanism_refs.csv	172	20	15	4.62	1.000
research_companies.csv	148	32	24	0.46	1.000
570-1.csv	544	143	85	8.48	1.000
6397-1.csv	800	24	22	8.63	1.000

6.2.1 Exemples de PFDs découvertes

Dataset t1.csv (employés gouvernementaux, 9 101 lignes) :

TABLE 7 – Top PFDs découvertes sur t1.csv (classique)

PFD		Support	Confidence
identity(Department)	→ Department Name	9 090	1.000
identity(Department Name)	→ Department	9 090	1.000
identity(Underfilled Job Title)	→ Assignment Cat.	998	0.996
numeric_prefix(Division, 1)	→ Assignment Cat.	2 232	0.995
identity(Division)	→ Department	8 527	0.987

La PFD `identity(Department) → Department Name` est une FD classique parfaite : chaque code de département détermine de manière unique le nom du département (confidence = 1.0, support = 9 090).

Dataset t2.csv (entreprises, 3 502 lignes) :

TABLE 8 – Top PFDs découvertes sur t2.csv (classique)

PFD		Support	Confidence
first_token(FAX)	→ COUNTRY	2 239	1.000
identity(NAME)	→ COUNTRY	1 994	1.000
identity(PHONE)	→ COUNTRY	1 952	1.000
identity(ADDRESS_1)	→ COUNTRY	1 774	1.000
last_token(FAX)	→ COUNTRY	1 605	1.000

Dataset mechanism_refs.csv (références pharmaceutiques, 9 536 lignes) :

La PFD `first_token(ref_id) → ref_type` (support = 1 776, confidence = 1.0) capture le fait que le premier token d'un identifiant de référence détermine son type (PubMed, DailyMed, Wikipedia).

6.3 Résultats de l'approche agentique

6.3.1 Comparaison globale sur t1.csv

Le tableau 9 présente la comparaison des 5 approches sur le dataset `t1.csv`.

TABLE 9 – Comparaison des approches sur t1.csv (9 101 employés)

Approche	PFDs	Parfaites	Candidats	Conf. moy	Supp. moy	Temps (s)
Classique	85	2	544	0.913	7 482	8.56
W1-Mistral	23	2	72	0.931	6 747	70.75
W2-Mistral	10	0	34	0.939	5 737	51.76
W1-Llama	12	0	40	0.915	7 804	62.52
W2-Llama	21	2	74	0.934	7 145	99.51

6.3.2 Analyse de la réduction de l'espace de recherche

L'un des avantages clés de l'approche agentique est la **réduction drastique de l'espace de recherche** :

TABLE 10 – Réduction de l'espace de recherche par rapport au classique

Approche	Candidats	Réduction	PFDs trouvées
Classique	544	–	85
W1-Mistral	72	–86.8%	23
W2-Mistral	34	–93.8%	10
W1-Llama	40	–92.6%	12
W2-Llama	74	–86.4%	21

Le Workflow 2 avec Mistral réduit l'espace de recherche de **93.8%**, ne validant que 34 candidats au lieu de 544, tout en maintenant une confiance moyenne supérieure (**0.939** vs. 0.913).

6.3.3 Suggestions des agents

Mistral (Workflow 1, t1.csv) a suggéré **9 transformations**, incluant :

- `first_token(Full Name)` : « le premier mot du nom complet est le prénom »
- `prefix(Date First Hired, 7)` : « les 7 premiers chiffres de la date d'embauche déterminent l'année »
- `identity(Department)`, `identity(Division)`, etc.

Llama (Workflow 1, t1.csv) a suggéré **5 transformations**, incluant :

- `first_token(Full Name)` : « le premier mot du nom complet est le prénom, souvent unique »
- `suffix(Division, 20)` : « les 20 derniers caractères de la division indiquent la spécialisation »
- `length(Position Title)` : « la longueur du titre peut classer les employés par niveau »

On note que les deux LLMs identifient correctement `first_token(Full Name)` comme transformation pertinente, mais divergent sur les transformations secondaires — Llama propose des transformations plus originales (`suffix`, `length`) tandis que Mistral se concentre sur les `identity`.

6.3.4 Candidats prioritaires (Workflow 2)

Mistral a priorisé 4 candidats :

1. `prefix(Department Name, 10)` → `Department` — priorité `high`
2. `prefix(Division, 35)` → `Division` — priorité `medium`
3. `prefix(Underfilled Job Title, 10)` → `Job Title` — priorité `medium`
4. `first_token(Position Title)` → `Job Role` — priorité `low`

Llama a priorisé 4 candidats :

1. `prefix(Full Name, 1)` → `Gender` — priorité `high`
2. `first_token(Department Name)` → `Department` — priorité `high`
3. `prefix(Date First Hired, 1)` → `Month of Hire` — priorité `medium`
4. `numeric_prefix(Division, 1)` → `Division Type` — priorité `medium`

Llama propose des candidats sémantiquement plus riches (`prefix(Full Name, 1)` → `Gender` est une véritable PFD originale), tandis que Mistral reste plus conservateur.

6.4 Comparaison Mistral vs. Llama

TABLE 11 – Comparaison des deux LLMs

Critère	Mistral 7B	Llama 3.1 8B
Nombre de suggestions (W1)	9	5
Conf. moyenne (W1)	0.931	0.915
Candidats explorés (W2)	34	74
Conf. moyenne (W2)	0.939	0.934
Temps moyen par workflow	~61s	~81s
Style de suggestions	Conservateur	Créatif
Réponse JSON	Plus fiable	Parfois verbeux

Mistral produit des résultats plus **précis** (confiance moyenne supérieure) avec un espace de recherche plus réduit (34 candidats en W2). **Llama** propose des transformations plus **diversifiées et originales** (`suffix`, `length`), mais explore un espace plus large (74 candidats en W2).

7 Discussion

7.1 Avantages de l’approche agentique

1. **Réduction massive de l’espace de recherche** : les workflows agentiques explorent 87–94% moins de candidats que l’approche classique, grâce au filtrage sémantique du LLM.
2. **Confidence moyenne supérieure** : en se concentrant sur des candidats sémantiquement pertinents, les approches agentiques obtiennent des PFDs de meilleure qualité (0.931–0.939 vs. 0.913 en classique).
3. **Interprétabilité** : chaque suggestion du LLM est accompagnée d’une **justification en langage naturel**, facilitant l’interprétation des résultats.
4. **Filtrage des dépendances triviales** : le LLM évite les PFDs sans sens sémantique (par exemple, `length(col) → col2` qui tiennent par coïncidence).

7.2 Limites et compromis

1. **Temps d’exécution** : l’inférence LLM ajoute 50 à 100 secondes de surcoût (8.5s en classique vs. 51–100s en agentique). Ce surcoût est fixe et ne dépend pas de la taille du dataset.
2. **Couverture** : l’approche classique découvre **85 PFDs** contre 10–23 en agentique. Certaines PFDs valides mais non intuitives sont manquées par le LLM.
3. **Déterminisme** : même avec une température très basse (0.1), les réponses LLM peuvent varier légèrement entre exécutions.
4. **Parsing JSON** : les LLMs ne respectent pas toujours le format JSON demandé, nécessitant un parsing robuste avec fallback (regex `\{[\s\S]*\}`).
5. **PFDs parfaites manquées** : W2-Mistral ne trouve aucune PFD parfaite (confiance = 1.0), alors que le classique en trouve 2. Le filtrage trop agressif du LLM peut exclure des dépendances valides.

7.3 Analyse du compromis qualité/exhaustivité

Le tableau 12 résume le compromis fondamental entre les approches.

TABLE 12 – Compromis qualité vs. exhaustivité

	Exhaustivité	Précision	Vitesse	Interprétabilité
Classique	***	**	***	*
Workflow 1	**	***	*	***
Workflow 2	*	***	**	***

L’approche classique excelle en **exhaustivité** et **vitesse**, tandis que les approches agentiques excellent en **précision** et **interprétabilité**. Le choix dépend du cas d’usage : l’exploration exhaustive est préférable pour l’audit de qualité de données, tandis que l’approche agentique convient mieux à la découverte ciblée de règles métier.

7.4 Reproductibilité

Tous les résultats sont reproductibles :

- Les LLMs sont exécutés **localement** via Ollama (pas de dépendance à des APIs cloud).
- Les paramètres ($K = 5$, $\theta = 0.85$) sont fixés dans les scripts.
- Les résultats sont sauvegardés en JSON avec horodatage.
- Le code est publié sur GitHub : <https://github.com/omarsoliman02/PFD-Discovery>

8 Conclusion

Ce projet a permis d'implémenter et de comparer deux paradigmes pour la découverte de PFDs :

1. Un **pipeline classique** en 5 étapes, explorant systématiquement toutes les combinaisons de transformations et de colonnes. Il découvre un grand nombre de PFDs (85 sur t1.csv) en un temps très court (8.5s).
2. Deux **workflows agentiques** utilisant des LLMs locaux (Mistral 7B et Llama 3.1 8B) pour guider la recherche. Le Workflow 1 (Feature-Enriched) réduit l'espace de recherche de $\sim 87\%$ et le Workflow 2 (Guided Search) de $\sim 94\%$, tout en maintenant ou améliorant la confiance moyenne.

Contributions principales :

- Implémentation complète en Python (1 400+ lignes) avec modules réutilisables.
- Comparaison systématique sur 14 datasets réels couvrant 3 domaines.
- Comparaison de 2 LLMs (Mistral vs. Llama) mettant en évidence des profils différents : Mistral est plus précis et compact, Llama est plus créatif et diversifié.
- Démonstration du compromis fondamental entre exhaustivité (classique) et précision sémantique (agentique).

Perspectives :

- Implémenter un **Workflow 3 (Agent-in-the-Loop)** avec boucle itérative de raffinement.
- Tester des LLMs plus puissants (Mistral 8x7B, Llama 70B) pour améliorer la qualité des suggestions.
- Explorer des métriques de qualité supplémentaires (interestingness, novelty) pour évaluer la pertinence sémantique.
- Étendre le système à la découverte de PFDs **multi-colonnes** ($X_1, X_2 \rightarrow Y$).

A Annexe : Guide d'utilisation

A.1 Installation

```

1 # Cloner le repository
2 git clone https://github.com/omarsoliman02/PFD-Discovery.git
3 cd PFD-Discovery
4
5 # Installer les dependances Python
6 pip install -r requirements.txt
7
8 # Installer Ollama (modeles locaux)
9 brew install ollama
10 brew services start ollama
11 ollama pull mistral          # Mistral 7B (~4.4 Go)
12 ollama pull llama3.1        # Llama 3.1 8B (~4.9 Go)

```

Listing 1 – Installation du projet

A.2 Exécution

```

1 # Approche classique sur tous les datasets
2 python3 experiments/run_classical.py
3
4 # Approche agentique (Mistral + Llama)
5 python3 experiments/run_agentic.py
6
7 # Comparaison sur un dataset spécifique
8 python3 experiments/compare_results.py --dataset t1
9
10 # Seulement Mistral, Workflow 1
11 python3 experiments/run_agentic.py --llm mistral --workflow 1

```

Listing 2 – Commandes d'exécution

B Annexe : Structure du code

```

1 PFD-Discovery/
2 |-- data/
3 |   |-- CHE/                (5 datasets chimie/biologie)
4 |   |-- DGOV/              (5 datasets gouvernance)
5 |   +-- pfd_validation/    (4 datasets validation)
6 |-- src/
7 |   |-- data_loader.py      Chargement CSV
8 |   |-- pattern_extraction.py 7 transformations
9 |   |-- pattern_grouping.py  Groupement par pattern
10 |   |-- candidate_generation.py Candidats X -> Y
11 |   |-- validation.py       Support + confidence
12 |   |-- generalization.py    Fusion de patterns
13 |   |-- classical_pipeline.py Pipeline 5 etapes
14 |   |-- llm_agent.py         Integration Ollama
15 |   |-- agentic_workflow.py  Workflow 1 + 2
16 |   +-- evaluation.py        Metriques

```



```
17 | -- experiments/  
18 | | -- run_classical.py      Script classique  
19 | | -- run_agentic.py       Script agentique  
20 | +-- compare_results.py     Comparaison  
21 | -- results/               Resultats JSON  
22 +-- requirements.txt
```

Listing 3 – Arborescence du projet